⭐ FULL GRAPH ALGORITHMS CHEATSHEET

🌟 1. BFS — Breadth-First Search

✅ Definition

BFS is a graph traversal algorithm that explores nodes **level by level**, starting from a source vertex and visiting all neighbors before going deeper.

Uses a **queue (FIFO)**.

✅ Applications

- Shortest path in **unweighted** graphs
- Checking **bipartiteness**
- Finding connected components
- Building BFS tree
- Level graphs in max-flow (Dinic)

✅ Time Complexity

Case	Time
Best	O(V + E)
Average	O(V + E)
Worst	O(V + E)

Space complexity: **O(V)**

✅ Important Notes

- Works correctly only if all edges have equal weight (or unweighted) for shortest paths
- Always produces the shortest number of edges
- Order depends on adjacency list order
- Cycles are not a problem (uses visited)

🌟 2. DFS — Depth-First Search

✅ Definition

DFS explores a graph by going **as deep as possible** before backtracking.

Uses a **stack** (implicit recursion or explicit stack).

✅ Applications

- Topological sorting
- Finding connected components
- Detecting cycles
- Finding articulation points & bridges
- Classifying edges (tree, back, forward, cross)
- DFS tree construction

✅ Time Complexity

Case	Time
Best	O(V + E)
Average	O(V + E)
Worst	O(V + E)

Space: **O(V)** stack depth

✅ Important Notes

- DFS is **not unique** — depends on adjacency ordering
- Handles cycles via visited
- Recursion may cause stack overflow on large graphs
- Analogous to **pre-order traversal** in trees
- Does **not** guarantee shortest paths

🌟 3. Topological Sort

✅ Definition

A linear ordering of vertices in a directed graph such that for every edge **u → v**,
u appears before v in the ordering.

Works **only on DAGs (Directed Acyclic Graphs)**.

✅ Applications

- Task scheduling
- Course prerequisite ordering
- Build systems
- Dependency resolution

✅ Algorithms

- DFS-based (post-order push)
- Kahn's Algorithm (BFS + in-degree)

✅ Time Complexity

Both algorithms run in:

O(V + E)

✅ Important Notes

- If the graph has a cycle → topological ordering **does not exist**
- Result may be **non-unique**
- DAG requirement is mandatory

🌟 4. Spanning Tree Algorithms (Kruskal & Prim)

🌿 What is a Spanning Tree?

A spanning tree of a graph:

1. Includes **all vertices**
2. Has **no cycles**
3. Contains exactly **V − 1 edges**
4. Is a **connected acyclic subgraph**

🌟 Kruskal's Algorithm

✅ Definition

A greedy algorithm that builds the Minimum Spanning Tree (MST) by selecting edges in **increasing order of weight**, ensuring no cycles using **DSU / Union-Find**.

✅ Steps

1. Sort edges by weight
2. Add the next lightest edge that doesn't form a cycle
3. Continue until MST has (V − 1) edges

✅ Time Complexity

- Sorting edges: **O(E log E) ≈ O(E log V)**
- DSU operations: **O(E α(V))**
- **Total: O(E log V)**

✅ Important Notes

- Works well for **sparse graphs**
- Requires a Union–Find structure
- If graph is disconnected → produces a **minimum spanning forest**

🌟 Prim's Algorithm

✅ Definition

Builds a Minimum Spanning Tree by repeatedly adding the **minimum-weight edge** connecting the tree to a new vertex.

Greedy approach, similar to Dijkstra.

✅ Time Complexity

Implementation	Time
Adjacency matrix	$O(V^2)$
Binary heap + adjacency list	$O(E \log V)$
Fibonacci heap	$O(E + V \log V)$

✅ Important Notes

- Best for **dense** graphs
- Requires graph to be **connected**
- Conceptually similar to Dijkstra's algorithm

🌟 5. Dijkstra's Algorithm

✅ Definition

Computes the shortest paths from a single source in a graph with **non-negative edge weights**.

Greedy algorithm using a **min-heap priority queue**.

✅ Time Complexity

Implementation	Time
Adjacency matrix	$O(V^2)$
Binary heap	$O(E \log V)$
Fibonacci heap	$O(E + V \log V)$

✅ Important Notes

- **Cannot** handle negative weights
- At most **E decrease-key** operations
- Distances only ever decrease (monotonic)
- Produces a shortest-path tree

🌟 6. Floyd–Warshall Algorithm

✅ Definition

A dynamic programming algorithm for finding **shortest paths between all pairs of vertices**.

Recurrence:

$$dist[i][j] = \min(dist[i][j],\ dist[i][k] + dist[k][j])$$

It checks whether paths improve by allowing vertex **k** as an intermediate.

✅ Time Complexity

- Best: **$O(V^3)$**
- Worst: **$O(V^3)$**
- Space: **$O(V^2)$**

✅ Important Notes

- Works with **negative weights**
- Does **not** work if there is a **negative cycle**
- Classic **bottom-up dynamic programming**
- Best for **small graphs** or dense graphs

🌟 7. Bellman–Ford Algorithm

✅ Definition

Computes the shortest paths from a single source in graphs with **negative edge weights**, and detects **negative cycles**.

⭐ Time Complexity

Case	Time
Best	O(E)
Average	O(VE)
Worst	O(VE)

⭐ Important Notes

- Handles **negative weights**
- Detects **negative cycles**
- Slower than Dijkstra
- Performs **V−1 relaxations** + cycle check

🌟 Final Summary Table

Algorithm	Purpose	Complexity	Notes
BFS	Shortest path in unweighted graphs	O(V+E)	Level-based
DFS	Exploration & structure analysis	O(V+E)	Non-unique, deep-first
Topological Sort	DAG ordering	O(V+E)	Only for DAGs
Kruskal	MST	O(E log V)	Best for sparse graphs
Prim	MST	O(E log V)	Best for dense graphs
Dijkstra	SSSP (no negative weights)	O(E log V)	Greedy
Floyd–Warshall	All-pairs shortest paths	$O(V^3)$	Negative weights OK
Bellman–Ford	SSSP (negative weights)	O(VE)	Detects negative cycles

Examples

⭐ 1) BFS — Breadth-First Search

(Поиск в ширину)

📌 Граф

```mathematica
      A
     / \
    B   C
   / \   \
  D   E   F
```

📌 **BFS(A)** — очередь и уровни

Очередь (FIFO):

1. Start: A
2. Visit neighbors B, C
3. Then D, E from B
4. Then F from C

📌 **Порядок BFS:**

A, B, C, D, E, F

📌 **Комментарий**

Идём **по уровням**.

⭐ 2) **DFS — Depth-First Search**

(Поиск в глубину)

📌 **Граф тот же**

```mathematica
      A
     / \
    B   C
   / \   \
  D   E   F
```

📌 **DFS(A)** — углубляемся максимально

1. A
2. B
3. D (глубже нельзя → назад)
4. E (назад)
5. назад к A
6. в C → F

📌 **Порядок DFS:**

A, B, D, E, C, F

📌 **Комментарий**

DFS ≈ как исследовать лабиринт — идём глубоко-вглубь.

⭐ 3) **Topological Sort**

(Топологическая сортировка)

📌 **Граф (DAG)**

```css
A → C → D
 \        ↑
  → B ----
```

📌 **Возможный топологический порядок:**

A, B, C, D

📌 Объяснение:

- A must come before B and C
- B before D
- C before D
- Граф **без циклов**, иначе невозможно.

⭐ 4) Kruskal's MST Algorithm

📌 Граф

```
    (1)
A-------B
|2      /3
|      /
|7    /4
C---D
    6
```

Рёбра:
A-B (1)
A-C (2)
B-D (3)
B-C (4)
C-D (6)
C-A (7)

📌 Шаги Краскала

Сортируем рёбра по весам:

1) A-B (1)

2) A-C (2)

3) B-D (3)

4) B-C (4)

5) C-D (6)

Добавляем, если **нет цикла**.

📌 Полученное MST:

A-B (1)

A-C (2)

B-D (3)

⭐ 5) Prim’s MST Algorithm

📌 Тот же граф, начинаем с A

```
    (1)
A-------B
|2       /3
|       /
|7    /4
C---D
    6
```

A connects cheapest to:

B (1), C (2)

Pick B first (1)

Then from {A,B}, next cheapest: C (2)

Then from {A,B,C}: D (3)

📌 MST:

A-B (1)

A-C (2)

B-D (3)

📌 Отличие от Краскала

Prim растёт **из одной точки**, Kruskal — **по рёбрам**.

⭐ 6) Dijkstra’s Algorithm

📌 Граф (неотрицательные веса)

```
A --1-- B --2-- C
 \      |      /
  4     7     1
   \    |    /
      ---- D
```

📌 Найти кратчайшие пути из A

Инициализация:

dist(A)=0, dist(B)=1, dist(D)=4

Дальше:

- через B → C = 1 + 2 = 3
- через D → C = 4 + 1 = 5 (не лучше)

📌 Итог:

A → B = 1

A → C = 3

A → D = 4

⭐ 7) Floyd–Warshall

(All-Pairs Shortest Paths)

📌 Матрица графа

```
    A   B   C
A   0   5   ∞
B   ∞   0   2
C   1   ∞   0
```

📌 Улучшаем через B, затем через C:

Через C можно улучшить A→B:

A→C→A→B?
A→C=∞ → не работает.

Но через C улучшается B→A:

B → C → A = 2 + 1 = 3

📌 Итоговая матрица кратчайших расстояний:

A 0 5 7

B 3 0 2

C 1 6 0

⭐ 8) Bellman–Ford Algorithm

📌 Граф с отрицательными весами

```
A --5--> B --3--> C
 \                ↑
  \--1--> D --(-4)
```

📌 Релаксации:

1. A→D = 1
2. D→C = 1 + (–4) = –3

3. A→B = 5
4. B→C = 8 (хуже чем –3)

📌 Кратчайший путь A→C:

A → D → C = –3

📌 Если после V–1 итераций что-то улучшается → negative cycle.